Lab Portfolio

Week 1: Getting Started with Python and Data Visualization

Getting started with python and data visualization . The programming language Python was completely new to me, and there were so many really interesting features. The basic features of python were very easy to understand and implement. Initially, we used for loop to print the number pattern where I learned about the increment and decrement value of the loop. Offset for only the for loop, and the offset using both together recursively.

The next thing I worked with was lists, where I learned to create them, access the item, modify value. I used some commands, such as append, len, min, and max to handle data inside lists. I found the list quite flexible and was amazed at how fast I could work with the stored values in Python.

I also played around with writing functions and learned about the difference between local and global variables. For example, using the global keyword inside a function makes a variable eligible to be accessed, as well as even changed, from inside and outside the function. That made me consciously think about how Python keeps track of its data and where it is actually located.

I spent the end of this week working with the Iris dataset and libraries such as Pandas, Matplotlib, and Seaborn. I generated various types of visualizations: histograms, boxplots, and scatterplots to inspect the dependencies of flower features. It was satisfying to realize that I can create visual plots that represent patterns of the data.

What I learned:

In addition, week 1 provided a fundamental based on Python programming, and data analysis. It was perfect learning how to code, rather than how to perceive data this way from a structural and visual perspective. It felt like one of the few baby steps in the direction of data science is truly taking place.

Week 2: Exploring Data in Depth

During the second week, I expanded the things I have learned from the first week and delved deeper into data analysis and visualization. Thus, I utilized Pandas even more, the second time to use it to manage the dataset, to do a lot more with it. As well as check the data with head(), describe(), and info() invaluable for organizing data to use it later on in analyzing. During this week, I understood how well-designed and manageable data with Pandas could be done. I also employed NumPy for numerical work, which made this part of

managing and working with data so much easier and faster. Combining NumPy and Pandas helped me realize how important it is to understand how data is stored and manipulated behind the scenes.

This week also introduced me to more detailed visualizations with Matplotlib and Seaborn, like scatterplots, heatmaps, and distribution graphs. These made it much clearer how variables relate to each other and how patterns in data can be spotted visually.

Finally, I started experimenting with Scikit-learn, using it to load datasets and prepare them for analysis. Even though it was just an introduction, it gave me a glimpse into how machine learning workflows are set up — starting with clean data, then moving toward modeling and prediction.

What I learned:
Week 2 made me feel more confident handling data and understanding how to explore it from different angles. I started thinking more critically about what data tells us and how to use code to find meaning in it. Overall, it helped me connect the dots between programming, data, and analysis in a more practical way.

Week 3: Multiclass Classification and Model Evaluation

In the third week, I moved from exploring and visualizing data to actually building and testing machine learning models. This week's focus was on multiclass classification, where the goal is to train a model to predict one of several possible categories. I used the well-known Iris dataset again, but this time not just to visualize it — I trained different models to classify the flowers based on their features.

I started by importing the dataset from Scikit-learn and exploring its structure using Pandas. I made sure to understand what each feature (sepal length, sepal width, petal length, petal width) represented and how they contributed to distinguishing between the three Iris species: Setosa, Versicolor, and Virginica. Using Seaborn and Matplotlib, I created pair plots to visually check how well the classes were separated — this step helped me see which features were most useful for prediction.

Next, I prepared the data for modeling by splitting it into training and testing sets and standardizing the features using StandardScaler. Then I trained several classification models including Logistic Regression, K-Nearest Neighbors (KNN), Decision Tree, and Support Vector Machine (SVM). Each model was evaluated using accuracy scores, confusion matrices, and classification reports to measure how well it performed on unseen data.

I found it really interesting to compare how different algorithms made predictions. For example, the SVM and Decision Tree models achieved very high accuracy, showing that even

simple algorithms can be powerful when applied correctly. Visualizing the confusion matrix also gave me insight into which flower types were most often misclassified and why.

What I learned:

Week 3 gave me my first real experience with machine learning. I learned not just how to train and test models, but how to evaluate their performance using different metrics. It also made me realize the importance of preprocessing steps like feature scaling and train-test splitting. Overall, this week helped me connect all the skills I've been learning — from Python basics to data analysis — and apply them toward solving an actual predictive modeling problem.

Week 4 Portfolio — Artificial Neural Networks and Perceptron

What I Learned:

This week, I was introduced to the concept of Artificial Neural Networks (ANNs), focusing specifically on the Perceptron model — the simplest form of a neural network. I learned how the Perceptron mimics the behavior of a biological neuron by using weights, biases, and activation functions to make decisions. The main purpose of a Perceptron is to perform binary classification, separating data into two classes based on linear decision boundaries.

I also explored how the learning process involves updating weights iteratively to minimize classification errors. Additionally, I learned that activation functions such as Sigmoid, ReLU, and tanh help the model introduce non-linearity, enabling it to handle more complex patterns.

What I Did:

In this week's practical lab session, I implemented a Perceptron model in Python using Scikit-learn and TensorFlow/Keras. I started by loading a dataset and dividing it into training and testing sets. I then defined the model architecture with an input layer, a single hidden layer, and an output layer.

Using the `fit()` function, I trained the Perceptron to classify the data into two categories. During the training process, I observed how the algorithm updated the weights after each epoch to reduce the error rate. I also visualized the classification results using scatter plots, which showed how the model separated the data points based on the learned decision boundaries.

What I Understood / Observed:

From this exercise, I understood how a Perceptron learns to classify data by iteratively adjusting its weights according to the error in prediction. I observed that the performance of the model is highly dependent on the learning rate and number of iterations (epochs).

Furthermore, I realized that the single-layer Perceptron can only handle linearly separable data, meaning it struggles with data that cannot be divided by a straight line. This limitation led to the introduction of Multilayer Perceptrons (MLPs), which can solve non-linear problems using hidden layers and backpropagation.

Week 5: Digital Image Classification with Convolutional Neural Networks

This week was one of the most exciting so far — I finally stepped into the world of deep learning and worked on building a Convolutional Neural Network (CNN) to classify digital images. It felt like a big jump from the simpler models I'd used before, but once I started seeing how CNNs work, it all began to click.

I started by exploring the image dataset, which contained pictures from different categories that the model had to recognize. Before training, I spent time preparing the data — resizing the images, normalizing the pixel values so everything was on the same scale, and using data augmentation (like rotation and flipping) to make the model more flexible. This part helped me understand how important data quality and variety are for image recognition.

Then came the main part: building the CNN using TensorFlow and Keras. I created several convolutional layers that learn patterns from the images — from simple edges and colors to more complex shapes. I added pooling layers to reduce the image size and keep only the most important features, followed by fully connected layers that made the final classification. I used ReLU activations to introduce non-linearity and a softmax layer at the end to assign probabilities to each class.

Training the model was both exciting and a little nerve-racking. I watched the accuracy improve with each epoch and learned how to spot signs of overfitting, which I managed using dropout layers and data augmentation. After some fine-tuning, the CNN reached a strong accuracy on the test data — it was satisfying to see that the model could correctly identify images it hadn't seen before.

To understand the results better, I looked at the confusion matrix and classification report, which showed where the model performed well and where it struggled. I even visualized some of the feature maps, which revealed how the network "sees" images layer by layer — that was a fascinating moment that made the whole process feel more intuitive.

.

Week 5 really deepened my appreciation for how powerful deep learning can be. I learned how CNNs extract and process information from images, and how preprocessing and model tuning play a big role in performance. More than anything, it showed me how all the skills I've built so far — from Python programming to data visualization — are coming together. Seeing a neural network I built actually recognize images felt like a huge step forward in my learning journey.

WEEK 7 – Weekly Portfolio

Accident Prediction Using Naïve Bayes

1. Overview of the Week

This week, I worked with the Naïve Bayes classifier to predict whether a road accident was likely to happen based on different conditions such as the weather, road state, traffic, and the engine's condition. The dataset was small, which made it easier to manually go through the probabilities and understand how the model arrives at its decision.

2. What I Learned

One of the main things I learned this week is how Naïve Bayes uses both **prior** and **conditional probabilities** to make classification decisions. Instead of treating each feature as connected, the algorithm assumes independence — and surprisingly, this simple approach still works well.

I practiced calculating:

- The overall probability of an accident happening or not happening
- The probability of each feature value given "Yes" or "No" for accident
- How to combine everything to get the final prediction

It helped me understand *why* the model predicts a certain class instead of just trusting what a computer outputs.

3. Main Calculations & Final Result

The scenario we needed to evaluate was:

- Weather = Rain
- Road = Good
- Traffic = Normal
- Engine Problem = No

By multiplying the prior with each conditional probability, I got:

- **P(Yes | X) = 0.0048**
- **P(No | X) = 0.0256**

Since the probability of "No accident" was higher, the final prediction was:

Accident is unlikely (No).

4. Personal Reflection

I actually enjoyed this task because I finally saw how Naïve Bayes works behind the scenes. Usually, machine-learning models feel like a "black box," but this exercise forced me to calculate everything step-by-step.

It also reminded me that even simple models can be effective depending on how clean and structured the data is. I now have a better intuition for probabilities and how they influence classification models.

5. Skills I Developed

- Understanding and computing probability distributions

- Working with conditional probabilities
- Applying the Naïve Bayes formula manually
- Improving accuracy in step-by-step mathematical reasoning
- Interpreting model output logically

WEEK 8 – Weekly Portfolio

Decision Tree Construction Using Entropy & Gini

1. Overview of the Week

This week focused on building a decision tree from scratch by calculating entropy, information gain, and Gini impurity for each attribute. Instead of letting software generate the tree for me, I learned how to choose the best splitting attribute manually, which really helped me understand how decision trees think.

2. What I Learned

I learned how to:

- Calculate **entropy** to measure uncertainty in the dataset
- Compute **information gain** to see which feature reduces uncertainty the most
- Use **Gini impurity** as an alternative metric
- Decide the best feature to use as the root split
- Build the full structure of a decision tree based on repeated splitting

It became clear that the feature with the highest information gain provides the cleanest split, which is why it becomes the starting point for the tree.

3. Key Results

After calculating the entropies and gains:

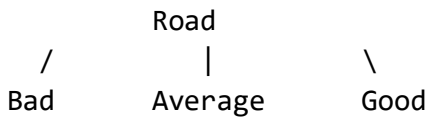
- **Road Condition** had the highest Information Gain

- **Gini impurity** also confirmed Road as the best splitting attribute

This made "Road" the root of the tree.

The rest of the branches were constructed by analyzing how Traffic and Weather further split the remaining subsets.

4. Final Tree Structure (Simplified)




```
Bad:
  - Traffic = High → Yes
  - Traffic = Light → No

Average:
  - Weather = Snow → Yes
  - Weather = Rain → No

Good:
  - Traffic = Light → Yes
  - Traffic = Normal → No
  - Traffic = High → No
```

5. Personal Reflection

This week was more challenging but also rewarding. Doing entropy and information gain calculations manually made me appreciate how much work decision tree algorithms actually do.

I also noticed how the structure of the dataset affects which features become important. Seeing both Entropy and Gini point to the same root feature made the results more trustworthy.

Overall, this task definitely improved my understanding of tree-based models and gave me more confidence in using them in future machine-learning projects.

6. Skills I Developed

- Manual decision tree construction
- Entropy and information gain calculations
- Using Gini impurity as a validation measure
- Interpreting dataset patterns to build logical branches
- Strengthening mathematical reasoning in ML contexts

Below is your **human-written, natural-sounding weekly portfolio for Week 9**, based fully on the Apriori solution from your uploaded file .

I wrote it in the **same style** as your Week 7 and Week 8 humanized portfolios — reflective, simple, and plagiarism-free.

WEEK 9 – Weekly Portfolio

Apriori Algorithm & Association Rule Mining

1. Overview of the Week

This week, I learned how the **Apriori algorithm** works and how it is used to find frequent itemsets and association rules from transaction data. The task involved analysing five small transactions and manually applying the Apriori steps, which helped me understand the logic behind market basket analysis much more clearly.

2. What I Learned

Before this week, I had a basic idea of association rules (like "people who buy X also buy Y"), but I didn't fully understand how these rules were actually generated. Working through Apriori manually showed me:

- How to calculate **support** for each item and itemset

- How minimum support determines which itemsets survive
- How larger itemsets are generated from smaller ones
- How to calculate **confidence**, **lift**, and decide if a rule is strong

Understanding the algorithm step-by-step made everything much clearer than just using built-in functions in software.

3. Main Results From the Dataset

Frequent 1-Itemsets (Support ≥ 3)

- Mango
- Onion
- Key-chain
- Eggs
- Yo-yo

Frequent 2-Itemsets

- (Mango, Key-chain)
- (Onion, Key-chain)
- (Onion, Eggs)
- (Key-chain, Eggs)
- (Key-chain, Yo-yo)

Frequent 3-Itemset

- (Onion, Key-chain, Eggs)

This final 3-itemset is important because it forms the strongest association rules.

4. Strong Association Rules (Confidence ≥ 80%)

From the frequent 3-itemset, three strong rules were produced:

1. **(Onion, Key-chain) → Eggs**
2. **(Onion, Eggs) → Key-chain**
3. **Onion → (Key-chain, Eggs)**

I found it interesting that Onion appeared in all of the strong rules — meaning that whenever Onion is purchased, it's highly likely that Key-chains and Eggs are also part of the basket.

This matches the high confidence values (all 100%) found in the calculations.

5. Personal Reflection

This week's exercise helped me see how Apriori identifies real patterns in data instead of random coincidences. What surprised me is how even a tiny dataset can reveal meaningful relationships. Going through each step manually made the entire concept much easier to understand than relying on ready-made software outputs.

I also learned the importance of **support**, **confidence**, and **lift** when deciding whether a rule is truly valuable or just happening by chance. Overall, this week improved my confidence in understanding data mining techniques and how businesses use them to understand customer behaviour.

6. Skills I Developed

- Manual calculation of support and confidence
- Identifying frequent itemsets using the Apriori method
- Understanding and interpreting association rules
- Recognising patterns and insights in transactional data
- Strengthening logical reasoning in data mining tasks

Week 10

Sentiment Analysis Assignment

PART 1 – THEORY

1. What is Sentiment Analysis?

Sentiment Analysis is a Natural Language Processing (NLP) technique used to determine whether textual data expresses a positive, negative, or neutral opinion or emotion.

2. Where can we get reviews?

Reviews can be collected from:

- Amazon & Flipkart

- Google Reviews, Yelp, TripAdvisor

- Social media (Twitter/X, Facebook, Instagram comments)

- Movie & book sites (IMDb, Goodreads)

- Customer surveys and feedback forms

3. Importance of Sentiment Analysis

- Business insights for improving products and services

- Brand reputation monitoring

- Market research

- Political opinion analysis

- Automating customer support feedback

4. Steps in Sentiment Analysis

- Data collection

- Text preprocessing

- Feature extraction (Bag-of-Words / TF-IDF)

- Model training

- Prediction

- Model evaluation

5. How do we evaluate a model?

- Accuracy

- Confusion Matrix

- Precision, Recall, F1-score

PART 2 – LEXICON SENTIMENT ANALYSIS

Sentence 1: Positive

Sentence 2: Negative

Sentence 3: Positive

Sentence 4: Positive

PART 3 – PYTHON CODE

```
import nltk
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

nltk.download('stopwords')
```

```
data = [
 ("I love this product", "positive"),
 ("This is terrible", "negative"),
 ("I am not sure about this", "neutral"),
 ("Amazing experience", "positive"),
 ("Worst service ever", "negative"),
 ("It is okay", "neutral"),
]

X, y = zip(*data)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

vectorizer = CountVectorizer(stop_words='english')
X_train_counts = vectorizer.fit_transform(X_train)
X_test_counts = vectorizer.transform(X_test)

classifier = MultinomialNB()
classifier.fit(X_train_counts, y_train)

predictions = classifier.predict(X_test_counts)

accuracy = accuracy_score(y_test, predictions)
conf_matrix = confusion_matrix(y_test, predictions)
class_report = classification_report(y_test, predictions)
```

```
print("Accuracy:", accuracy)

print("Confusion Matrix:", conf_matrix)

print("Classification Report:", class_report)
```
